CANDEX HANDBOOK

UI / UX

L'apparence, le ressenti et les patterns d'interaction de l'app mobile Candex — design tokens, thèmes, bibliothèque de composants, navigation et conventions UX.

Rédigé par
Eliott Barbet

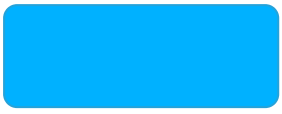
Version 0.23.0
novembre 2026
candex.app

UI / UX

L'interface de Candex est une app mobile **Ionic 8 + Angular 20**. Ce handbook couvre le côté design et interaction ; pour la structure du code front-end, voir Front-end.

Design tokens

Les couleurs, la typographie et les surfaces sont définies une seule fois sous forme de **variables CSS Ionic** et réutilisées partout, pour que la marque reste constante sur chaque écran.



primary

#00B1FF

Bleu Candex — actions principales, accents

secondary

#ffffff

Blanc



tertiary

#033C44

Teal profond — emphase secondaire



success

#2dd36f

État succès



warning

#ffc409

État avertissement



danger

#eb445a

État danger / erreur

Chaque couleur est livrée avec ses variantes `-shade`, `-tint` et `-contrast`, pour que le texte respecte toujours son fond. La typographie est **Rubik**, auto-hébergée depuis `assets/fonts` (Regular, Medium, Bold et le reste de la famille) et branchée via `--ion-font-family`, donc l'app s'affiche identiquement hors-ligne et sur toutes les plateformes.

Typographie

La police de l'app est **Rubik**, auto-hébergée depuis `assets/fonts` et définie via `--ion-font-family`. Trois graisses sont chargées et portent toute la hiérarchie : **Regular (400)** pour le

corps de texte, **Medium (500)** pour l'emphase et les labels, et **Bold (700)** pour les titres et le texte fort.

Les tailles de texte suivent la **typographie intégrée d'Ionic** (l'app importe `typography.css`) plutôt qu'une échelle sur mesure : une base de **16px (1rem)**, les titres décroissant de `<h1>` à `<h6>` sur l'échelle par défaut d'Ionic. En bref, l'app possède la *famille* ; Ionic possède les *tailles* — ce qui garde le texte constant et respecte les réglages d'accessibilité (dynamic type) de la plateforme.

Conventions de style

Les styles sont écrits en SCSS et suivent quelques règles strictes, pour que la base reste uniforme :

- **En cascade, toujours** — les styles sont écrits en SCSS imbriqué qui s'appuie sur la cascade, pas en sélecteurs plats et répétés.
- **Une échelle d'espacement fixe** — marges, paddings, gaps et décalages n'utilisent que **2, 4, 8, 16, 32** px. `12` et `24` sont autorisés, mais uniquement dans des cas rares et précis.
- **Des variables globales pour les valeurs partagées** — couleurs, polices et tailles de texte viennent toujours de variables globales (les design tokens), jamais de littéraux en dur.
- **Pas de CSS flottant** — pas de style inline dans les templates et rien de stylé hors d'un fichier `.scss` ; chaque règle vit dans le SCSS.
- **La consistance avant tout** — la même chose se fait de la même façon partout.

Voici à quoi cela ressemble, extrait de `update-required.component.scss` : un bloc racine par composant, les enfants imbriqués avec `&__`, les couleurs uniquement via les variables du thème (light et dark fonctionnent donc sans effort), les espacements sur l'échelle :

```
.update-required {
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 32px 24px;
  background: var(--ion-background-color);

  &__icon {
    margin-bottom: 16px;
    color: var(--ion-color-primary);
  }

  &__text {
    margin: 0 0 24px;
    color: var(--ion-color-medium);
  }
}
```

Thèmes — Light, Dark, System

L'app propose trois thèmes via un `ThemeService` : **Light**, **Dark** et **System** (qui suit le réglage de l'appareil). Le choix est appliqué en posant un attribut `theme` sur `<body>`, et les blocs de tokens `body[theme=light]` / `body[theme=dark]` échangent la palette en dessous — chaque composant réagit automatiquement puisqu'il lit les mêmes variables.

Bibliothèque de composants

Au-dessus des composants d'Ionic (boutons, cards, listes, modales, tabs) s'ajoute un ensemble de **composants partagés** qui portent les patterns propres à l'app :

- **Contenu** — `drink-card` affiche une canette de façon constante dans le Dex et les listes.
- **Feedback** — `banner` et `error-banner` remontent des messages inline ; `form-errors` affiche la validation au niveau du champ sous les inputs.
- **Tâches ciblées (modales)** — `avatar-cropper-modal`, `fullscreen-image-modal`, `report-problem-modal` et les flux de compte Google (`google-account-modal`, `google-`

`link-prompt-modal`, `google-photo-choice-modal`) gardent une seule tâche en vue à la fois.

- **Auth** — `social-auth-buttons` fournit les points d'entrée de connexion sociale.

Les icônes viennent d'**Ionicons**, le set fourni avec Ionic.

Navigation

L'app repose sur un **shell à trois tabs** (`tabs.page`) :

- **Dex** — la collection, et le tab d'atterrissage par défaut.
- **Search** — une action au tap qui ouvre la recherche plutôt qu'une page de tab statique.
- **Profile** — le compte et les réglages de l'utilisateur.

Les pages sont des **composants standalone chargés en lazy** (`loadComponent`), donc chaque écran se charge à la demande. Hors des tabs, une **stack d'auth** porte le parcours de connexion — `welcome`, `register`, `login`, `verify-email`, `two-factor` et `complete-profile` — et des routes dédiées gèrent les vues de détail et de réglages (`drink-details/:drinkId`, `settings`). L'authentification elle-même est traitée dans Sécurité.

Conventions d'interaction & de contenu

Quelques patterns reviennent partout :

- **Des modales pour le travail ciblé** — recadrer un avatar, voir une image en plein écran, lier un compte Google ou signaler un problème se font dans une modale, en gardant l'écran sous-jacent intact.
- **Validation inline, au niveau du champ** — les formulaires affichent les erreurs à côté du champ fautif via `form-errors`, pas en alertes intrusives.
- **Des banners pour le statut** — les messages transitoires et d'erreur apparaissent en banners plutôt qu'en dialogues bloquants.

Internationalisation

L'UI est entièrement traduite avec **ngx-translate** : les chaînes sont externalisées dans `en.json` et `fr.json` et rendues via le pipe `| translate` (`tabs.dex`, `tabs.search`, …), pour que l'anglais et le français restent synchronisés et qu'une nouvelle langue se résume à ajouter un fichier de locale.

Pour aller plus loin

- Le code derrière ces écrans → Front-end
- À quoi sert l'app → Comprendre Candex
- Connexion et permissions → Sécurité